



N-ATLAS Kit

Technical Documentation: Architecture, Setup & Usage

A developer toolkit for N-ATLAS, Nigeria's multilingual Large Language Model: Python SDK, CLI, REST API, evaluation harness, interactive playground and the Şõra reference application.

Item	Detail
Programme	National AI Innovation Challenge (NAIC) — Innovation & Enterprise Track
Problem statement	PS1: Developer Infrastructure
Team	Team N-ATLAS Kit — Lead: Adeyemi Jamiu Adegbenro
Repository	https://github.com/ADEYEMIjamiu/natlas-kit (MIT licence, CONTRIBUTING.md)
Version / date	v0.1.0 · October 2026
Model	NCAIR1/N-ATLaS (Llama-3-8B based), GGUF Q4_K_M quantization
Languages	Yoruba, Hausa, Igbo, Nigerian-accented English (+ Nigerian Pidgin heuristics)

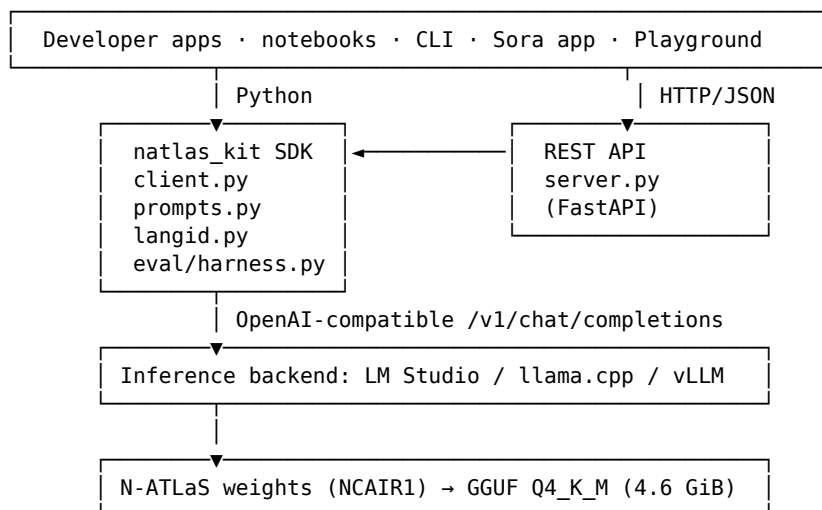
1. Overview

N-ATLAS is openly released, but a Nigerian developer who wants to ship a product on it still has to answer the same questions alone: how do I run an 8-billion-parameter model without a GPU, how do I call it from my app, which prompts work in Yoruba, Hausa and Igbo, and how do I know if the output is good enough? N-ATLAS Kit answers these once, in open source, so teams can spend their time on their product.

Design goals

- **Laptop-first:** runs N-ATLAS fully offline on a 16 GB laptop (no GPU, no paid API), which matters where bandwidth and power are costly.
- **Backend-agnostic:** speaks the OpenAI-compatible protocol, so the same code works with LM Studio, llama.cpp, vLLM, Ollama or a hosted endpoint.
- **Zero-dependency core:** the SDK uses only the Python standard library; server, demo and test extras are optional.
- **Measurable:** every deployment can be benchmarked on Nigerian-language tasks with one command.
- **Honest evaluation:** separate seed (tuning) and held-out (never tuned) test sets.

2. Architecture



Module	Responsibility
natlas_kit/client.py	NAtlas class: HTTP transport, retries/errors, usage accounting, high-level tasks (chat, stream, translate, detect_language, classify, classify_scam), health checks.
natlas_kit/prompts.py	Prompt templates tuned for N-ATLAS, few-shot translation examples, scam-detection system prompt, JSON parsing, rule-based red-flag and safe-signal scoring and blending.
natlas_kit/langid.py	Instant lexical language ID (Yoruba, Hausa, Igbo, Pidgin, English) using script characters and marker words; falls back to N-ATLAS when unsure.
natlas_kit/eval/harness.py	Benchmark runner: scam detection (accuracy, precision, recall, F1, per-language), translation (chrF, tone-insensitive chrF), language ID; JSON reports.
natlas_kit/cli.py	natlas command: doctor, chat, translate, scam, eval, serve.
natlas_kit/server.py	FastAPI REST API with Swagger docs and an in-memory request log (/stats) for usage evidence.

Module	Responsibility
apps/scam_checker/	Şõra Streamlit app (scam checker with feedback logging) and Playground page.
tests/test_kit.py	Offline pytest suite against a fake OpenAI-compatible server (no model needed).

3. Setup

3.1 Requirements

Component	Minimum	Tested
Hardware	8 GB RAM (Q4 model $\approx$ 4.6 GiB) · ~6 GB free disk	Apple M4 MacBook Air, 16 GB RAM
OS	macOS, Windows or Linux	macOS
Python	3.9+	3.14
Backend	Any OpenAI-compatible server	LM Studio (lms CLI), port 1234
Model access	Hugging Face account; accept terms at huggingface.co/NCAIR1/N-ATLaS	—

3.2 Get and serve the model

Create a Hugging Face read token and store it in a local `.env` file (excluded from git by `.gitignore`). Then:

```
echo "HF_TOKEN=hf_xxx" > .env
bash setup_model.sh
# downloads N-ATLaS Q4_K_M GGUF -> ~/.lmstudio/models/,
# starts LM Studio server on :1234, loads it as model id "n-atlas", runs a Yoruba test
```

Alternatives: `llama-server -m N-ATLaS.Q4_K_M.gguf --port 1234 (llama.cpp)` or `vllm serve NCAIR1/N-ATLaS` on a GPU, then set `NATLAS_BASE_URL`.

3.3 Install and verify the kit

```
git clone https://github.com/ADEYEMIjmiu/natlas-kit && cd natlas-kit
python3 -m pip install -e ".[server,demo,dev]"
natlas doctor          # server reachable? model loaded? greets in YO/HA/IG
pytest                 # 8 offline tests
```

3.4 Configuration

Variable	Default	Purpose
NATLAS_BASE_URL	<code>http://localhost:1234/v1</code>	OpenAI-compatible endpoint serving N-ATLAS
NATLAS_MODEL	<code>n-atlas</code>	Model identifier on that endpoint
NATLAS_API_KEY	<code>not-needed</code>	Bearer token for hosted endpoints
HF_TOKEN (.env)	—	Only used by <code>setup_model.sh</code> to download weights

4. Usage

4.1 Python SDK

```
from natlas_kit import NATlas
nt = NATlas()                                # or NATlas(base_url=..., model=...)

nt.chat("Şé o lè şàlàyé ohun tí AI jẹ́ ní şókí?")
for tok in nt.stream([{"role": "user", "content": "Bawo ni?"}]): print(tok, end="")
nt.translate("Take this medicine twice daily", target="ha")      # en/yo/ha/ig/pcm
nt.detect_language("How far, I don reach house")                 # -> "pcm"
nt.classify("Light never come for 3 days", ["electricity", "water", "roads"])
```

```
nt.classify_scam("Your BVN has been blocked, send your PIN")
# {'label': 'scam', 'risk': 86, 'reasons': [...], 'rule_flags': [...],
#  'safe_signals': [], 'advice': '...', 'language': 'en'}
```

Method	Returns	Notes
chat(message, system=None, history=None)	str	Replies in the user's language by default
stream(messages)	iterator[str]	Server-sent events token streaming
translate(text, target, source='auto')	str	Few-shot prompts; temperature 0
detect_language(text)	code	Hybrid lexical + N-ATLAS fallback
classify(text, labels, instruction="")	label	Zero-shot classification
classify_scam(text)	dict	Model verdict blended with red-flag / safe-signal rules: risk = 0.7·model + 0.3·rules
health()	dict	Endpoint reachability and loaded models

4.2 Command line

```
natlas doctor
natlas chat # interactive, streaming
natlas translate "Where is the hospital?" --to ig
natlas scam "Oga, EFCC don flag your account. Pay clearance fee"
natlas eval --tasks scam,translate,langid --out eval_results
natlas eval --tasks scam,langid --scam-data my_data.jsonl
natlas serve --port 8000 # REST API
```

4.3 REST API

Endpoint	Body	Response
GET /health	—	status, models, languages
POST /v1/translate	{"text", "target", "source"}	{"translation"}
POST /v1/scam-check	{"text"}	label, risk, reasons, rule_flags, advice, language
POST /v1/detect	{"text"}	{"language"}
POST /v1/chat	{"message", "system"}	{"reply"}
GET /stats	—	request count and recent latency log

```
curl -X POST localhost:8000/v1/scam-check -H "Content-Type: application/json" \
-d '{"text": "È ku oriire! È ti jẹ ẹbun N300,000. È san owo N5,000"}'
```

4.4 Apps

- **Şora scam checker** (streamlit run apps/scam_checker/app.py): paste an SMS, WhatsApp message or job advert in any of 5 languages; shows risk score, reasons and advice; thumbs-up / thumbs-down feedback is written to session_log.csv as real-user validation evidence.
- **Playground** (sidebar page): chat, translate, detect language and custom classification, each showing the equivalent Python snippet to copy.

5. N-ATLAS Integration Details

- **Model:** official open weights NCAIR1/N-ATLAS (Llama-3-8B fine-tune; 4 safetensors shards, ~16 GB in BF16), accessed with an approved Hugging Face token.
- **Quantization:** GGUF Q4_K_M (4.58 GiB loaded) for CPU/Apple-silicon inference; loads in ~18 s on an M4 laptop.

- **Serving:** LM Studio exposes /v1/chat/completions and /v1/models; the SDK uses only these two routes, so any compatible server works.
- **Prompting:** system prompts identify the model as a Nigerian assistant; translation uses two few-shot pairs per language (not drawn from the test sets) and a 'keep the sentence type' instruction; classification and scam detection use temperature 0 and JSON output with robust parsing.
- **Hybrid safety layer:** the scam detector blends the N-ATLAS verdict with transparent rule-based red flags (BVN/NIN, PIN/OTP requests, upfront fees, urgency, links) and safe signals ("do not share", receipts, balances) so the 8B quantized model stays reliable and every verdict is explainable.

6. Evaluation

All runs: N-ATLaS Q4_K_M via LM Studio on an Apple M4, 16 GB. Reports are committed under eval_results/. The seed set was used to tune prompts; the held-out set was written afterwards and **never used for tuning**.

Task (metric)	Seed set: baseline	Seed set: after tuning	Held-out set (unseen)
Scam detection (F1)	0.93 (recall 1.00, precision 0.87)	1.00	1.00 (30/30)
Scam detection (accuracy)	0.917	1.00	1.00
Language ID (accuracy)	0.722	0.944	1.00 (30/30)
EN→YO/HA/IG translation (chrF)	53.1 (YO 30.2, HA 62.1, IG 66.9)	54.3 (YO 51.6, HA 57.5, IG 53.9)	—
Translation, tone-insensitive chrF	—	63.6	—
Set size	36 messages / 15 pairs	36 / 15	30 messages (16 scam, 14 legit)

Interpretation and limitations

- Test sets are small and curated; results show the kit works end to end across five languages, not production-grade accuracy. Real-user messages collected through Şõra are the next validation step.
- Yoruba translation scores are sensitive to tone marks; the tone-insensitive chrF is reported because everyday Yoruba is often typed without them.
- Translation references are short everyday sentences and require native-speaker review; contributions are invited via CONTRIBUTING.md.
- Pidgin is not an official N-ATLAS language; it is supported through prompts and lexical rules.

7. Testing & Quality

- pytest: 8 offline tests using a fake OpenAI-compatible HTTP server (health, chat, translation clean-up, scam blending, input validation, metrics, JSON parsing, full eval run).
- natlas doctor: live smoke test against the real model.
- Secrets: .env is git-ignored; the token is never logged or printed.

8. Deployment Options

Scenario	How
Single developer, offline	LM Studio or llama.cpp on a laptop + SDK/CLI

Scenario	How
Team / startup API	llama.cpp or vLLM on a cloud VM; natlas serve behind HTTPS; set NATLAS_API_KEY
Web/mobile app	Call the REST API from any language; Swagger docs at /docs
CI quality gate	Run natlas eval in CI and fail the build if F1 or chrF drops

9. Roadmap

- JavaScript/TypeScript SDK and a hosted public demo endpoint.
- Larger community-reviewed benchmarks (FLORES-style translation, sentiment, NER) in all three languages.
- Retrieval-augmented generation (RAG) helpers for Nigerian government, health and finance documents.
- Speech pipeline adapters for the official N-ATLAS ASR service.
- Fine-tuning starter kit (LoRA) that exports back into the eval harness.

Repository: <https://github.com/ADEYEMIjmiu/natlas-kit> · Licence: MIT (kit code); N-ATLaS weights under NCAIR's licence · Contact: Adeyemi Jamiu Adegbenro, speeditdg@gmail.com